



Tyk Onboardin g

Rahmat - Sr. Customer Solutions Architect



Module 1

Tyk Operator

Understanding API Management in Kubernetes with Tyk Operator

Imperative vs Declarative

Imperative model (HOW)

You describe a sequence of instructions to manipulate the state of the system to reach your desired state

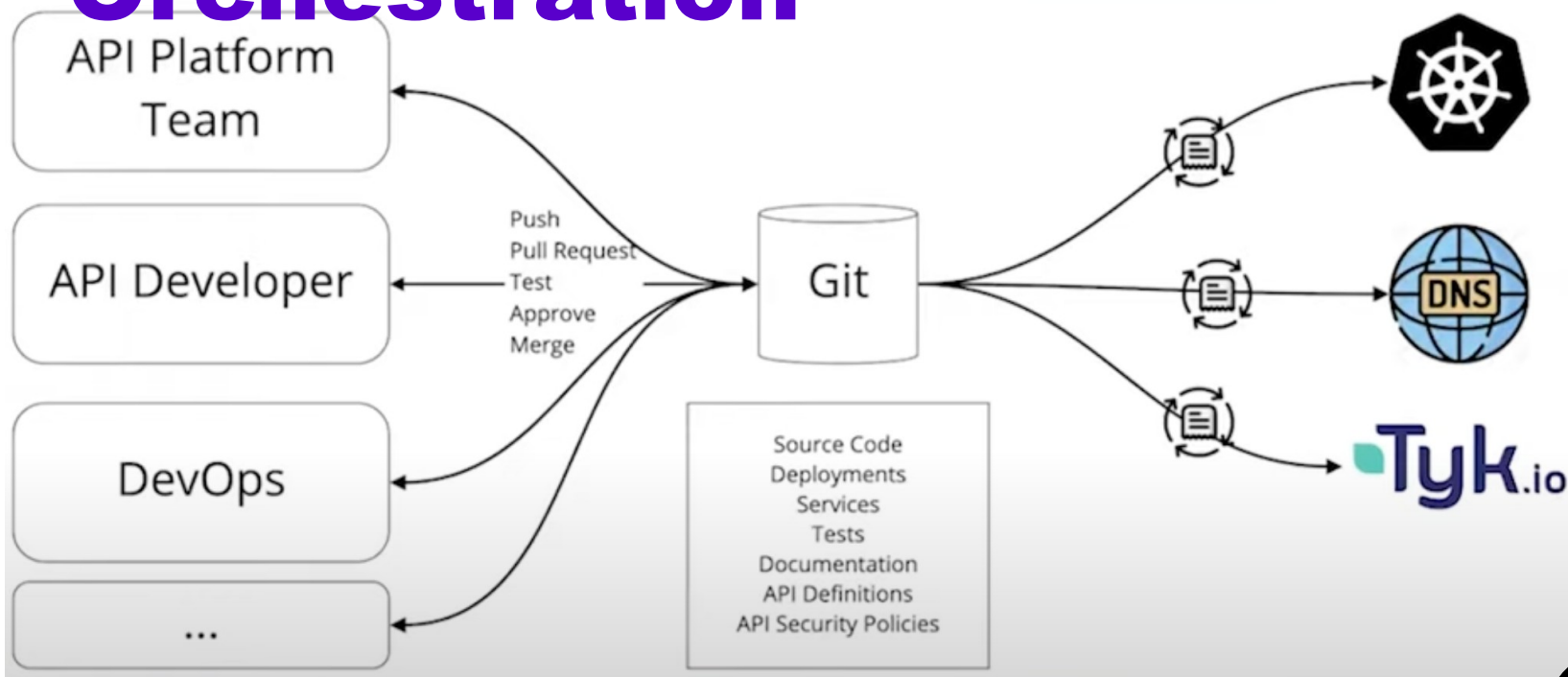
```
kubectl create deployment my-application  
--image=my-app:latest  
kubectl scale deployment my-application --replicas 3  
kubectl annotate deployment my-application  
environment=prod
```

Declarative model (WHAT)

You describe what you want to be achieved, as opposed to how to get there

```
kubectl apply -f my-application.yaml
```

Kubernetes Orchestration



Tyk Operator

- Tyk Operator within Kubernetes allows you to manage API lifecycles **declaratively**
- Native Kubernetes operator
- Define and manage APIs as code
- Deploy, update, and secure APIs using the same declarative configuration approach

Custom Resources

- Custom Resources (CRs) extend the Kubernetes API.
- They allow you to define and manage application-specific configurations.
- CRDs make Kubernetes highly extensible by enabling domain-specific objects.
- Defined using Custom Resource Definitions (CRDs), which specify the schema and structure of the resource

Tyk Operator

- Tyk Operator uses CRDs to manage API configurations declaratively.
- These custom resources align with different API types and security policies.
- Benefits:
 - GitOps-ready
 - Version-controlled API configurations
 - Integrated with Kubernetes CI/CD workflows

Custom Resources

```
apiVersion: tyk.io/v1alpha1
kind: TykOasApiDefinition
metadata:
  name: petstore
spec:
  tykOAS:
    configmapRef:
      name: tyk-oas-api-config # Metadata name of the ConfigMap resource
                                # that stores the Tyk OAS API Definition
      namespace: default      # Metadata namespace of the ConfigMap
                                # resource
      keyName: oas-api-definition.json # Key for retrieving Tyk OAS API
                                      # Definition from the ConfigMap
```


Tyk Operator CRDs

Kind	Group	Version	Description
TykOasApiDefinition	tyk.tyk.io	v1alpha1	Defines configuration of Tyk OAS API Definition object
ApiDefinition	tyk.tyk.io	v1alpha1	Defines configuration of Tyk Classic API Definition object
TykStreamsApiDefinition	tyk.tyk.io	v1alpha1	Defines configuration of Tyk Streams
SecurityPolicy	tyk.tyk.io	v1alpha1	Defines configuration of security policies . Operator supports linking ApiDefinition custom resources in SecurityPolicy's access list so that API IDs do not need to be hardcoded in the resource manifest.
SubGraph	tyk.tyk.io	v1alpha1	Defines a GraphQL federation subgraph .
SuperGraph	tyk.tyk.io	v1alpha1	Defines a GraphQL federation supergraph .
OperatorContext	tyk.tyk.io	v1alpha1	Manages the context in which the Tyk Operator operates, affecting its overall behavior and environment. See Operator Context for details.

Kubernetes Controllers

- **Definition:** A controller is a control loop that watches the state of your cluster and makes changes to move the current state toward the desired state.
- **Part of Kubernetes Core:** Many controllers are built into Kubernetes by default (e.g., Deployment controller, ReplicaSet controller, Job controller).
- **Function:** It continuously reconciles resources by reading their spec and taking action to match the desired state.
- **Example:** If you create a Deployment with 3 replicas, the ReplicaSet controller ensures that 3 pods are always running.

Kubernetes Operator

- **Definition:** An operator is a custom controller that manages complex, domain-specific applications or workloads.
- **Built on Controllers:** Operators use controllers to manage Custom Resources (CRDs).
- **Extends Kubernetes:** Operators understand the application lifecycle — including installation, upgrades, backups, configuration changes, etc.
- **Human Expertise in Code:** Acts as a Kubernetes user
- **Example:** The Tyk Operator manages API definitions and security policies by watching custom resources like `TykOasApiDefinition`.

Reconciliation

- Reconciliation is a key design pattern in Kubernetes controllers.
 - It ensures the actual state of the system matches the desired state.
 - Tyk Operator uses this pattern to keep Kubernetes objects in sync with Tyk.
- Reconciliation is event-driven — it happens when something changes in your cluster.
- Triggers:
 - A Kubernetes object is created, updated, or deleted
 - The Tyk Operator pod is restarted
 - The Operator's cache expires (typically ~10 hours)

Important:

Reconciliation does not react to changes made directly in Tyk!

Reconciliation

What Happens During Reconciliation?

Tyk Operator compares:

- Desired State (from Kubernetes object)
- Actual State (fetched from Tyk)

If there's a drift, it performs one of three actions:

- CREATE → Exists in K8s but not in Tyk
- UPDATE → Differs in Tyk and K8s (via hash check)
- DELETE → Deleted in K8s but still in Tyk

Drift Detection

- Drift Detection is built into reconciliation.
- If someone:
 - Deletes an API from Tyk manually
 - Changes a config directly via Tyk Dashboard
- Tyk Operator restores the Kubernetes-defined version.
- This protects against:
 - Accidental updates
 - Unauthorized changes

Best Practice:

Use read-only access to Tyk in production environments.

Summary

- Tyk Operator is a Kubernetes Controller that manages Tyk Custom Resources (CRs) such as:
 - API Definitions
 - Security Policies
- How it works:
 - Developers define API configurations using Custom Resources
 - Tyk Operator ensures the desired state is continuously reconciled with the Tyk Gateway or Dashboard
 - Handles CREATE, UPDATE, and DELETE operations to align the actual state with the desired state
- Goal:
 - Achieve and maintain a consistent, declarative API infrastructure through Kubernetes-native workflows

Module 2

Operator Deep Dive

Diving into Tyk Operator

Authentication with Tyk Dashboard

Tyk Operator as a System User

- Acts as a system user for the Tyk Dashboard
- Bound by Organization ID and RBAC rules

Startup Behavior

- On startup, Tyk Operator retrieves credentials from:
 - `tyk-operator-conf` Kubernetes Secret
 - Environment Variables

Key or Environment Variable	Description
<code>TYK_MODE</code>	"ce" for OSS or "pro" for licensed users
<code>TYK_URL</code>	URL of Tyk Gateway or Dashboard API
<code>TYK_ORG</code>	Organization ID of Operator user
<code>TYK_AUTH</code>	API key of Operator user

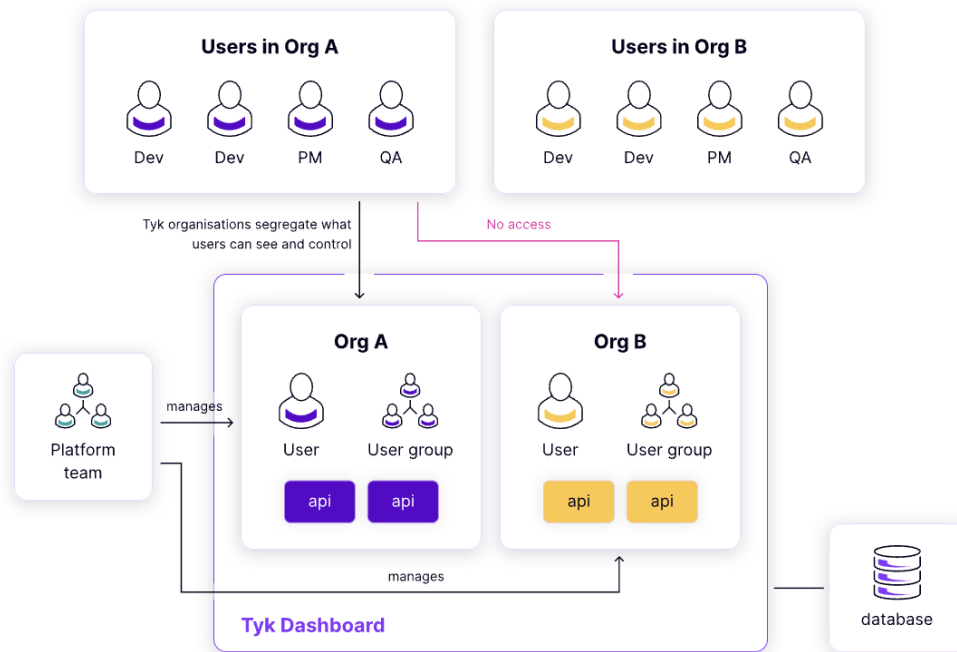
Multi-Tenancy in Tyk

Run multiple isolated teams or departments on a single Dashboard instance.

Each organization is fully independent, with its own:

- API Definitions
- API Keys
- Users
- Developers
- Domain

Ideal for complex enterprise structures where teams operate independently under one infrastructure.



Multi-Tenancy in Tyk Operator

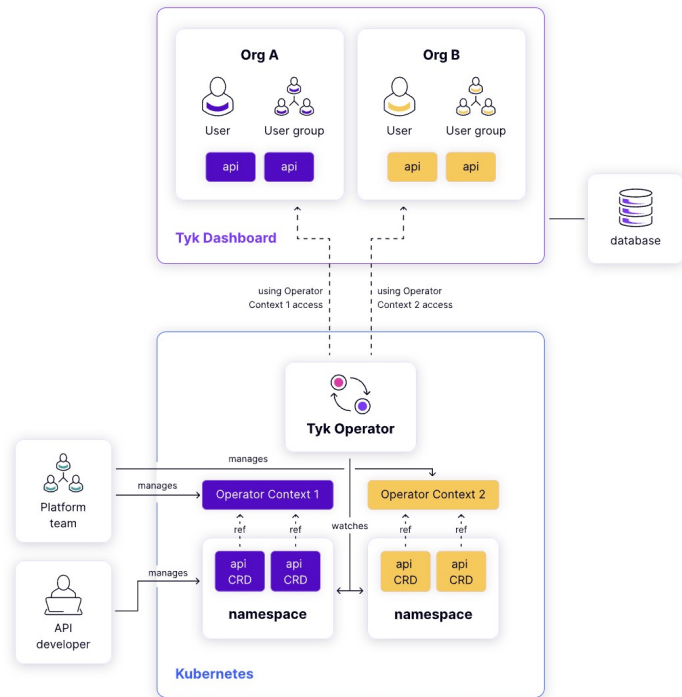
`OperatorContext` allows isolated API management per team or department within a shared Tyk Operator instance.

What `OperatorContext` Defines:

- Tyk Dashboard to interact with
- Organization for API management
- User identity for API requests
- Environment where the Operator runs

You can reference `OperatorContext` in:

- `ApiDefinition`
- `TykOasApiDefinition`
- `TykStreamsApiDefinition`
- `SecurityPolicy`



Multi-Tenancy in Tyk Operator

How `OperatorContext` Works:

Scenario	Behavior
Not Defined	Defaults to credentials from <code>tyk-operator-conf</code> or env vars
Defined but Not Referenced	Still uses default credentials; <code>OperatorContext</code> is ignored
Defined and Referenced	Uses specified credentials; allows isolated, team-specific API control

TLS Certificate Management

Tyk Operator simplifies how you manage TLS certificates for APIs in Kubernetes.

Traditional Approach:

- Manually upload certificates to Tyk
- Reference via Certificate ID
- Manual updates = high overhead and risk of downtime

With Tyk Operator:

Use certificates stored as Kubernetes secrets

→ Reference directly in your CRDs

Supported in:

`ApiDefinition`

`TykOasApiDefinition`

`SecurityPolicy`, etc.

TLS Certificate Management

Tyk Operator simplifies how you manage TLS certificates for APIs in Kubernetes.

Traditional Approach:

- Manually upload certificates to Tyk
- Reference via Certificate ID
- Manual updates = high overhead and risk of downtime

With Tyk Operator:

Use certificates stored as Kubernetes secrets

→ Reference directly in your CRDs

Supported in:

`ApiDefinition`

`TykOasApiDefinition`

`SecurityPolicy`, **etc.**

Module 3

Installing Tyk Operator

Configuring and Installing Tyk Operator

Installing and Configuring Tyk Operator

Pre-requisites:

1. Tyk deployment - Tyk Cloud or Self-managed
2. Policy ID matching configuration:
 - a. Dashboard configuration set to `true`:
 - i. `allow_explicit_policy_id`
 - ii. `Enable_duplicate_slugs`
 - b. Gateway configuration set to `true`:
 - i. `Policies.allow_explicit_policy_id`
3. User credentials from the Dashboard
4. Tyk Operator user above should write access to APIs, Certificates, Policies etc.
5. Turn off write access for all other users

Admission Webhooks

Admission webhooks are HTTP callbacks triggered during the Kubernetes API request lifecycle.

They let you:

- Validate custom resources (`ValidatingAdmissionWebhook`)
- Mutate or inject defaults into resources (`MutatingAdmissionWebhook`)

Commonly used by Operators and Controllers to enforce policies and set defaults.

- Ensures only valid configurations are accepted
- Allows default values to be injected automatically
- Improves security, consistency, and automation

Cert Manager

- Kubernetes requires all webhook communication to be over HTTPS.
- The webhook server must serve a valid certificate.
- Kubernetes must trust the certificate used by the webhook.

This is where cert-manager comes in.

- cert-manager is a Kubernetes add-on that automates:
 - Issuing TLS certificates
 - Renewing expiring certs
 - Storing them in secrets
- Works with issuers like Let's Encrypt, HashiCorp Vault, internal PKI.

You can disable the webhook and/or manually generate TLS certificates

Example use case:

- Air-gapped environments
- Clusters with restricted access

Installing Tyk Operator

2 options to install Tyk Operator:

1. Tyk's Umbrella Helm Charts
2. Stand-alone Helm Chart

Tyk's Umbrella Helm Charts:

- You can install Tyk Operator alongside other Tyk components in the Tyk Helm Chart by setting:
`global.components.operator = true`
- A license key is required for the Tyk Operator.
- You can provide it via:
 - Helm value: `global.license.operator`
 - Or a Kubernetes secret:
 - `global.secrets.useSecretName` → **Secret must contain: OperatorLicense**
- If using `global.secrets.useSecretName`, the value in `global.license.operator` will be ignored.
- Ensure the referenced secret is properly configured.

Installing Tyk Operator via Umbrella Helm Chart

You can install Tyk Operator alongside other Tyk components in the Tyk Helm Chart by setting:

```
global.components.operator = true
```

- `tyk-operator-conf` will have been created with the following keys by default:
 - `TYK_OPERATOR_LICENSEKEY`
 - `TYK_AUTH` - Tyk Dashboard API Access Credentials
 - `TYK_MODE`
 - `TYK_ORG` - Organization ID
 - `TYK_URL`
- If the credentials embedded in the `tyk-operator-conf` are ever changed or updated, the `tyk-operator-controller-manager` pod must be restarted to pick up these changes.

Installing Tyk Operator via Stand-Alone Helm Chart

You can install CRDs and Tyk Operator using the stand-alone Helm Chart by running the following command:

```
$ helm repo add tyk-helm https://helm.tyk.io/public/helm/charts/  
$ helm repo update  
  
$ helm install tyk-operator tyk-helm/tyk-operator -n tyk-operator-system
```

This process will deploy Tyk Operator and its required Custom Resource Definitions (CRDs) into your Kubernetes cluster in tyk-operator-system namespace.

Secret/envVars Key-Values

Key	Mandatory	Example Value	Description
TYK_OPERATOR_LICENSEKEY	Yes	<JWT_ENCODED_LICENSE_KEY>	Tyk Operator license key
TYK_MODE	Yes	pro	"ce" for Tyk Open Source mode, "pro" for Tyk licensed mode.
TYK_URL	Yes	http://dashboard-svc-tyk-tyk-dashboard.tyk.svc:3000 ↗	Management URL of Tyk Gateway (Open Source) or Tyk Dashboard
TYK_AUTH	Yes	2d095c2155774fe36d77e5cbe3ac963b	Operator user API key.
TYK_ORG	Yes	5e9d9544a1dcd60001d0ed20	Operator user ORG ID.
TYK_TLS_INSECURE_SKIP_VERIFY	No	true	Set to <code>"true"</code> if the Tyk URL is HTTPS and has a self-signed certificate. If it isn't set, the default value is <code>false</code> .
WATCH_NAMESPACE	No	foo,bar	Comma separated list of namespaces for Operator to operate on. The default is to operate on all namespaces if not specified.
WATCH_INGRESS_CLASS	No	customclass	Define the ingress class Tyk Operator should watch. Default is <code>tyk</code>
TYK_HTTPS_INGRESS_PORT	No	8443	Define the ListenPort for HTTPS ingress. Default is <code>8443</code> .
TYK_HTTP_INGRESS_PORT	No	8080	Define the ListenPort for HTTP ingress. Default is <code>8080</code> .

Module 4

Managing APIs with Tyk Operator

Advanced Features of Tyk Operator

Onboarding a Tyk OAS API

1. Prepare the Tyk OAS API Definition

a. Create an OAS-compliant file (e.g., `oas-api-definition.json`)

b. Include Tyk extensions:

```
"x-tyk-api-gateway": {  
  "info": { "name": "Petstore", "state": { "active": true } },  
  "upstream": { "url": "https://petstore.swagger.io/v2" },  
  "server": { "listenPath": { "value": "/petstore/", "strip": true } }  
}
```

Tip:

Generate and export this from the Tyk Dashboard > API > Actions > View API Definition

Onboarding a Tyk OAS API

2. Create Kubernetes ConfigMap

```
kubectl create configmap tyk-oas-api-config \  
  --from-file=oas-api-definition.json -n tyk
```

Important Notes:

Max size: 1MiB

Avoid `kubectl apply` due to 256KB annotation limit

Use `kubectl create` or `kubectl replace`

Onboarding a Tyk OAS API

3. Define TykOasApiDefinition Resource

Manifest (tyk-oas-api-definition.yaml):

```
apiVersion: tyk.tyk.io/v1alpha1
kind: TykOasApiDefinition
metadata:
  name: petstore
spec:
  tykOAS:
    configmapRef:
      name: tyk-oas-api-config
      namespace: tyk
      keyName: oas-api-definition.json
```

```
kubectl apply -f tyk-oas-api-
definition.yaml
```

Onboarding a Tyk OAS API

4. Verify & Test the API

Check Status:

```
kubectl get tykoasapidefinition petstore
```

Test Endpoint:

```
curl "TYK_GATEWAY_URL/petstore/store/inventory"
```

Onboarding a Tyk OAS API

5. Updating an API

To make any changes to your API configuration, update the OAS file in your `ConfigMap` and then re-apply the `ConfigMap` using `kubectl replace`:

```
kubectl create configmap tyk-oas-api-config --from-file=oas-api-  
definition.json -n tyk --dry-run=client -o yaml | kubectl replace -f -
```



Hands-On Workshop

Onboarding an OAS API using Operator

Onboarding a Tyk OAS API

Create a file called `oas-api-definition.json` with your API definition, including the Tyk extension `x-tyk-api-gateway`. For example:

```
{
  "info": {
    "title": "Petstore",
    "version": "1.0.0"
  },
  "openapi": "3.0.3",
  "components": {},
  "paths": {},
  "x-tyk-api-gateway": {
    "info": {
      "name": "petstore",
      "state": {
        "active": true
      }
    },
    "upstream": {
      "url": "https://petstore.swagger.io/v2"
    },
    "server": {
      "listenPath": {
        "value": "/petstore/",
        "strip": true
      }
    }
  }
}
```

Onboarding a Tyk OAS API

Step 2: Create a ConfigMap from the OAS file

Upload the API definition into Kubernetes as a ConfigMap:

```
kubectl create configmap tyk-oas-api-config --from-file=oas-api-definition.json
```

- `tyk-oas-api-config` is the name of the ConfigMap.
- `-n tyk` specifies the namespace; replace `tyk` if your Tyk Operator runs in a different namespace.

Onboarding a Tyk OAS API

- **Step 3: Create the TykOasApiDefinition Custom Resource manifest**
- Create a YAML file called `tyk-oas-api-definition.yaml`:

```
apiVersion: tyk.tyk.io/v1alpha1

kind: TykOasApiDefinition

metadata:

  name: petstore

  namespace: tyk

spec:

  tykOAS:

    configmapRef:

      name: tyk-oas-api-config

      namespace: tyk

    keyName: oas-api-definition.json
```


Onboarding a Tyk OAS API

Step 4: Apply the TykOasApiDefinition resource

```
kubectl apply -f tyk-oas-api-definition.yaml
```

This tells the Tyk Operator to create the API based on your OAS definition.

Onboarding a Tyk OAS API

Step 5: Verify API creation

Check the status of your API:

```
kubectl get tykoasapidefinition petstore
```

Expected output shows your API details and sync status, like:

NAME	DOMAIN	LISTENPATH	PROXY.TARGETURL	ENABLED	SYNCSTATUS
petstore		/petstore/	https://petstore.swagger.io/v2	true	Successful

Onboarding a Tyk OAS API

Step 6: Test the API

You can now send a request to your Tyk Gateway to test the API. Replace `TYK_GATEWAY_URL` with your actual gateway URL:

```
curl "http://TYK_GATEWAY_URL/petstore/store/inventory"
```

Onboarding a Tyk OAS API

Summary of commands:

```
# Create ConfigMap with your API OAS file
```

```
kubectl create configmap tyk-oas-api-config --from-file=oas-api-definition.json -n tyk
```

```
# Apply the TykOasApiDefinition manifest
```

```
kubectl apply -f tyk-oas-api-definition.yaml
```

```
# Verify the resource status
```

```
kubectl get tykoasapidefinition petstore -n tyk
```

```
# Test the API (replace TYK_GATEWAY_URL with your gateway URL)
```

```
curl "http://TYK_GATEWAY_URL/petstore/store/inventory"
```

Securing a Tyk OAS API

1. Update Tyk OAS Definition

Modify existing Tyk OAS API Definition to include the API key authentication configuration.

OAS definition stored in a file named `oas-api-definition.json`

ConfigMap named `tyk-oas-api-config` in the `tyk` namespace

Modify your Tyk OAS API Definition `oas-api-definition.json` as follow:

Securing a Tyk OAS API

```
components: {  
  "securitySchemes": {  
    "petstore_auth": {  
      "type": "apiKey",  
      "name": "Authorization",  
      "in": "header"  
    }  
  }  
},  
"security": [  
  {  
    "petstore_auth": []  
  }  
],  
"paths": {},  
"x-tyk-api-gateway": {  
  "info": {  
    "name": "Petstore",  
    "state": {  
      "active": true  
    }  
  },  
  "upstream": {  
    "url": "https://petstore.swagger.io/v2"  
  },  
  "server": {  
    "authentication": {  
      "enabled": true,  
      "securitySchemes": {  
        "petstore_auth": {  
          "enabled": true  
        }  
      }  
    }  
  },  
  "listenPath": {
```

`components.securitySchemes:`
defines the authentication method (in this case, `apiKey` in the header).

`security:` Applies the authentication globally to all endpoints.

`x-tyk-api-gateway.server.authentication:`
Tyk-specific extension to enable the authentication scheme.

Securing a Tyk OAS API

Update the ConfigMap with the new Tyk OAS API Definition:

```
kubectl create configmap tyk-oas-api-config --from-file=oas-api-definition.json -n tyk --dry-run=client -o yaml | kubectl replace -f -
```

Verify the changes:

```
kubectl get tykoasapidefinition petstore -o yaml
```

Test the API Endpoint:

```
curl -v "TYK_GATEWAY_URL/petstore/store/inventory"
```



Hands-On Workshop

Updating and Securing an API using Tyk Operator

Securing a Tyk OAS API

Step 1: Update your `oas-api-definition.json`

Changes from the previous example:

1. Add API key authentication to your OAS under `components.securitySchemes`
2. Add global security requirement in `security`
3. Enable authentication in the Tyk extension `x-tyk-api-gateway.server.authentication`

Securing a Tyk OAS API

Step 2: Update the ConfigMap in Kubernetes

Run the following command to update the existing ConfigMap (`tyk-oas-api-config`) with the new definition:

```
kubectl create configmap tyk-oas-api-config --from-file=oas-api-definition.json --dry-run=client  
-o yaml | kubectl replace -f -
```

- This command creates a new ConfigMap YAML from your file and pipes it into `kubectl replace` to update the existing ConfigMap.
- Replace `tyk` with your namespace if different.

Securing a Tyk OAS API

Step 3: Tyk Operator automatically reconciles the change

Since the existing `TykOasApiDefinition` resource references this ConfigMap, the Tyk Operator will detect the change and update the API configuration accordingly.

No need to manually re-apply the `TykOasApiDefinition` resource unless you want to change something there.

Securing a Tyk OAS API

Step 4: Verify the API update status

Check the TykOasApiDefinition resource status:

```
kubectl get tykoasapidefinition petstore -o yaml
```

Look for:

```
status:
```

```
  latestTransaction:
```

```
    status: Successful
```

```
    time: "2024-09-16T11:48:20Z"
```

Securing a Tyk OAS API

Step 5: Test your secured API

Try calling the API without an API key:

```
curl -v "http://TYK_GATEWAY_URL/petstore/store/inventory"
```

- Replace `TYK_GATEWAY_URL` with your actual Tyk Gateway URL.
- You should get **401 Unauthorized** because the API now requires a key.

Module 5

Advanced Tyk Operator

Advanced Features of Tyk Operator

Control Kubernetes Ingress Resources

- **Ingress** is a Kubernetes resource that defines how external HTTP/S traffic should be routed to internal services.
- An **Ingress Controller** is the engine that actually reads Ingress rules and applies them.
- The Tyk Operator acts like a smart Ingress Controller by:
 - Reading existing Ingress resources
 - Creating corresponding APIs in Tyk Gateway
 - Applying API management features like:
 - Authentication
 - Rate limiting
 - Monitoring
 - Transformations
- You don't have to manually define each API in Tyk—it generates them based on your Ingress resources and a template.

Key Concepts

Annotations

Inside your Ingress YAML, you use annotations like:

```
annotations:  
  kubernetes.io/ingress.class: tyk  
  tyk.io/template: myapideftemplate  
  tyk.io/template-kind: ApiDefinition
```

- `ingress.class: tyk`: tells Kubernetes to use Tyk Operator
- `template`: the name of a Tyk `ApiDefinition` or `TykOasApiDefinition` that acts as a config template
- `template-kind`: which type of template is being referenced

Key Concepts

How Tyk Operator Handles Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: httpbin-ingress
  annotations:
    kubernetes.io/ingress.class: tyk
    tyk.io/template: myapideftemplate
    tyk.io/template-kind: ApiDefinition
spec:
  tls:
    - hosts:
        - myingress.do.poc.tyk.technology
      secretName: httpbin-ingress-tls
  rules:
    - host: myingress.do.poc.tyk.technology
      http:
        paths:
          - path: /httpbin
            pathType: Prefix
            backend:
              service:
                name: httpbin
                port:
                  number: 8000
```

Field in Ingress	Becomes in Tyk
host	API custom domain
secretName	TLS cert in Tyk
path	API listen path
backend.service.name	Upstream service URL in Tyk
tyk.io/template	Base config template for API behavior

Key Concepts

The referenced template (`myapideftemplate`) is a Tyk API configuration, like this:

```
apiVersion: tyk.tyk.io/v1alpha1
kind: ApiDefinition
metadata:
  name: myapideftemplate
  labels:
    template: "true" # Important!
spec:
  use_keyless: false
  auth:
    auth_header_name: Authorization
    auth_type: jwt
  rate_limit:
    rate: 100
    per: 60
```

This template might define:

- Auth type (e.g., JWT, OAuth)
- Rate limits
- Headers
- Plugins

Tyk will copy this template, and only override:

- Domain
- Path
- TLS cert
- Upstream service

Key Concepts

Naming Convention:

- `<namespace>-<ingress_name>-<hash(host + path)>`
- Example: `default-httpbin-ingress-78acd160d`

Automatic Syncing:

When you update the Ingress (e.g., add new paths, change services), Tyk Operator:

- Detects the change
- Updates the corresponding APIs in Tyk
- Keeps everything in sync without manual effort

Key Concepts

Ingress Path Matching

Kubernetes allows three types of path matches:

PathType	Example Path	Request URL	Will Match?
Exact	/foo	/foo/	✗
Prefix	/foo/	/foo, /foo/	✓
Prefix	/foo	/foobar	✗